

ADAPTIVE SPARSE MEMORY LANGUAGE MODELS

Anonymous authors

Paper under double-blind review

ABSTRACT

Sparse memory layers offer finer-grained sparsity, which has been shown to improve language modeling performance. But querying these layers is costly, complicating large-scale training. Leveraging the key-value memory interpretation of Transformer MLPs, we propose learning to sparsely activate its key-value pairs through a gating mechanism. We first motivate our approach by showing that intermediate MLP activations have low effective rank relative to their width, suggesting that sparse computation could reduce this redundancy. We show that finegrained mixtures of experts achieve better use of the intermediate dimension, by compress input token representations into narrower spaces with less redundancy, which may explain why they can match dense models while using fewer active intermediate dimensions. This motivates learning to select only the intermediate coordinates needed for the current input token. To achieve this, we revisit the *hard-concrete distribution*, making it input-dependent and relaxed on the left side, yielding fully differentiable, adaptive, and cost-effective sparse gating. Our method, **ZeroNeuron (ZoN)**, trains effectively, matching dense model performance with lower estimated training compute.

1 INTRODUCTION

Increasing model size improves performance but also raises training costs. The question is how to train large models efficiently at smaller computational cost. Sparsity, particularly MoEs, has emerged as an effective solution for training large language models economically. In these models, the transformer’s MLP is replaced by larger matrices, but only parts of them are activated for each input token. MoEs can be viewed as activation sparsity applied to groups of channels: only a subset of these groups is activated for each token. Keeping the total matrix size and sparsity fixed while reducing the group size yields finer-grained sparsity, which has been shown to improve language modeling. Taking this granularity to the extreme leads to an architecture similar to a sparse memory layer (He, 2024; Lample et al., 2019) roughly corresponding to one expert per channel.

A memory layer stores trainable key-value pairs (k_i, v_i) , with $k_i \in \mathbb{R}^{d_q}$ and $v_i \in \mathbb{R}^n$, but only a subset of pairs is used for computation for a given input token. Given an input $x \in \mathbb{R}^d$, a query network first produces $q(x) \in \mathbb{R}^{d_q}$. Each key receives a score $s_i = q(x)^\top k_i$, measuring how well it matches the query. The layer keeps only the k keys with the largest scores. We denote the indices of these selected keys by $\mathcal{I}$ and apply $w = \text{Softmax}((s_i)_{i \in \mathcal{I}})$ to obtain their weights. Finally, the corresponding values are combined into the output $m(x) = \sum_{i \in \mathcal{I}} w_i v_i$. Thus, the query determines which values are retrieved and how much each contributes.

While appealing for achieving finer-grained sparsity, sparse memory layers face several challenges. Selecting keys becomes expensive as the memory grows. Approaches such as PKM and Ultra-Sparse reduce this cost through decomposition, but TopK selection still adds computational cost as the candidate set grows. When the memory is distributed across GPUs, tokens need to be sent to the GPUs hosting their selected memory slots. These data transfers make sparse memory layers communication-bound, especially when the memory spans many GPUs. Moreover, the discrete selection made by TopK is not differentiable, which can make routing harder to optimize at fine-grained high sparsity. Finally, TopK uses a fixed selection budget: every token activates the same number of key-value pairs, regardless of the difficulty of the task.

We go beyond standard sparse memory designs and TopK with the following contributions:

1. Leveraging the key-value memory interpretation of the MLP, we propose to make it sparse by learning to activate only the key-value pairs needed for each token.
2. We motivate this approach by observing that MLP intermediate activations mostly lie in a subspace much smaller than the full available space. We show that granular MoE experts compress input representations into narrower spaces which occupy more fully the available dimensions. These observations motivate learning to select which intermediate coordinates to activate.
3. To select the needed key-value pairs for each token, we revisit stretching methods to learn sparse gates in a fully differentiable way. The sparse gating is adaptive and applied point-wise, which is advantageous in distributed training as it allows each GPU to activate its local memory slots without needing token representation dispatch as in TopK-based routing. This enables retaining the communication pattern of a dense tensor-parallel MLP.

The next section describes the key-value memory view of the MLP, and provide an analysis of rank structure of the MLP in dense models but also in sparse models. We then describe the design of the sparse gating in Section, and finally the experimental results in Section.

2 CONTEMPLATING THE MLP

2.1 TRANSFORMER’S MLP AS A KEY-VALUE MEMORY

The MLP is defined by two linear projections $\mathbf{W}_k \in \mathbb{R}^{d_{\text{model}} \times d_{\text{inter}}}$, $\mathbf{W}_v \in \mathbb{R}^{d_{\text{inter}} \times d_{\text{model}}}$:

$$\text{MLP}(\mathbf{x}) = \sigma(\mathbf{x}\mathbf{W}_k)\mathbf{W}_v. \quad (1)$$

where σ is a non-linear activation function and $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ is the input token representation. Modern approaches usually implement the gated variants of this but let’s keep it simple. A possible interpretation of the MLP is to consider it as a key-value memory layer (Geva et al., 2021; Dai et al., 2022; Meng et al., 2023). $\mathbf{W}_k\mathbf{x}$ scores how well the input matches each key. The activation function σ transforms these scores into weights, which are then used to aggregate the corresponding value entries in $\mathbf{W}_v$. Usually, the dimension d_{inter} , named *intermediate dimension*, is much larger than d_{model} , often $4\times$, so it provides a large keys set. What kind of information is stored in this memory is still an active open question. (Geva et al., 2021) found that those key-value pairs act as lexical dictionaries, where some keys are associated with specific lexical tokens. But this lexical association is confounded to real-world knowledge in some cases. Such that when the key “The capital of France is” is associated with the value “Paris”, it goes beyond lexical association but reflects a knowledge.

If the MLP can be seen as a memory layer, it is arguable that this memory can be sparse, since a given input may not need to activate all the key-value pairs. It is conceivable that an input needs only to retrieve some needed key-value pairs. If so, one could design a sparse activation gating that predicts ahead of time which key-value memory pairs are needed for a given token representation. But to motivate this approach, we first need to examine whether the intermediate activations contain redundancy that a sparse gating could exploit. If so, we need then to design such sparse gating function.

2.2 THE EFFECTIVE DIMENSIONALITY OF THE MLP

We analyze the eigenstructure of the MLP, both in dense and sparse models.

Mesuring the rank We measure the effective dimensionality of the intermediate activations relative to their allocated width d_{inter} in dense and MoE transformers. We collect the output activations of $\mathbf{K}$ before the nonlinearity using a calibration dataset $\mathcal{D}$. For any input token $\mathbf{x} \in \mathcal{D}$, we define $\mathcal{K} = \{\mathbf{k} = \mathbf{W}_k \mathbf{x}\}_{\mathbf{x} \in \mathcal{D}}$ with $\mathbf{k} \in \mathbb{R}^{d_{\text{inter}}}$. We estimate the covariance matrix from these activations as $\Sigma = \mathbb{E}[\mathbf{k}\mathbf{k}^\top] - \mathbb{E}[\mathbf{k}]\mathbb{E}[\mathbf{k}]^\top$. We then compute the eigenvalues of this covariance matrix and sort them in decreasing order, $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$, where $d = d_{\text{inter}}$. We analyze the cumulative energy ratio $R(k)$, which is the proportion of total activation variance explained by the top- k eigenvalues:

$$R(k) = \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^d \lambda_j}, \quad k \in \{1, \dots, d\}.$$

We report $R@p$, with $p \in [0, 1]$, defined as the smallest fraction k/d such that $R(k) \geq p$. This ratio tells us how the activation energy is spread. If the activations are low rank, $R(k)$ reaches p more quickly, which means only a few dimensions are needed to represent most of the activation energy. For example, if $R@1.0 = 0.4$, then about 40% of the dimensions are needed to represent all the activation energy. In practice, because of numerical precision and small eigenvalue tails, reaching exactly 100% can be unstable. We therefore use $p = 0.99$ in our experiments. Overall the metric used is related to effective rank and stable rank, but provide more flexibility by varying p .

Table 1: We check biases in the energy ratio measurement on Qwen3-30B-A3B. n denotes routed tokens per expert and r denotes Ratio@0.99 (defined in the text); μ denotes the mean, $\tilde{n}$ the median routed tokens per expert, and ρ the correlation.

Metric	Value
Mean Ratio@0.99 across experts	0.8318
Mean routed tokens per expert	126,988.05
Median routed tokens per expert	96,432.50
ρ amount of routed tokens vs. ratio	0.0799

ple, Qwen3-30B-A3B has a total intermediate dimension of 98,304, organized into 128 subspaces of 768 dimensions each. Each token is projected on 8 of these subspaces. Hence, the higher energy ratio of MoEs reflects fuller use of its smaller intermediate subspaces. Since experts receive different numbers of tokens, we also check whether this affects the measured ratio. For Qwen3-30B-A3B, Table 1 negligible linear correlation between the number of routed tokens and the ratio ($\rho = 0.0799$). These observations motivate an approach where the MLP intermediate dimension is large but each token only selects its needed subspace.

Observation 1: The MLP has low effective intermediate dimension

Most activation energy is concentrated in a subspace with far fewer dimensions than the allocated intermediate space.

Observation 2: MoEs use smaller subspaces more fully

MoEs maintain a large total intermediate space, but each input token representation is projected into smaller subspaces. Activations occupy a larger fraction of the dimensions within these subspaces.

Results . Figure 1a shows the percentage of dimensions needed to represent 90% of the energy ratio. We can see that all dense models have low effective dimension. This is true also for very low dimensional models such as Monad. The linear projection already limits the covariance rank to at most $\min(d_{\text{model}}, d_{\text{inter}})$, so expanding to $d_{\text{inter}} > d_{\text{model}}$ necessarily leaves linear dependencies among the intermediate coordinates. Hence, the intermediate activations of dense models contain a lot redundancy. As shown in Figure 1b, finegrained sparse MoEs avoid this by projecting the input representation in different smaller subspaces. For exam-

3 LEARNING TO ADAPTIVELY SELECT THE NEEDED KEY-VALUE PAIRS

We showed that dense MLP activations occupy only part of the available intermediate space. MoEs instead maintain a large total intermediate space, but each input token is projected into smaller subspaces that are used more fully. This motivates selecting individual memory slots for each input token. To avoid the fixed selection budget and token dispatch of TopK-based sparse memory layers, we propose a new structured sparse gating that learns to zero-out intermediate coordinates for each input token.

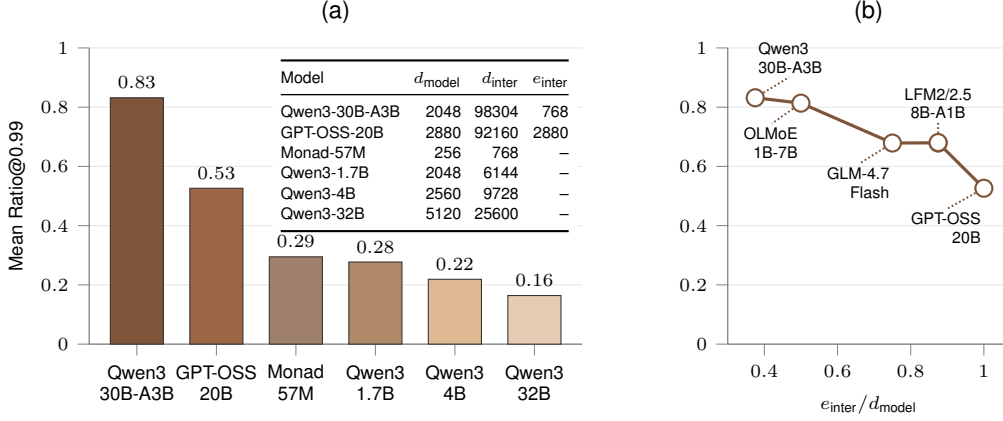


Figure 1: Mean $R@0.99$ across models (left) and its relationship to relative expert width across six MoEs (right). Higher $R@0.99$ means that more activation components are needed to represent 99% of the activation energy. The table gives dimensions for each model. For MoEs, e_{inter} is the intermediate dimension per expert and d_{inter} is the number of experts times e_{inter} . MoE ratios are measured within each expert, relative to e_{inter} . The line is a visual guide, averaging the two Liquid models at their shared width ratio; both measurements are shown as markers.

3.1 ZERO NEURON SPARSE ACTIVATION GATING

We revisit the hard-concrete distribution (Louizos et al., 2018), originally proposed to learn binary masks $\mathbf{z}$ for pruning. The core idea is to concentrate probability mass at 0 and 1 through a “stretching” mechanism. It starts with a continuous relaxation of Bernoulli sampling, producing values between 0 and 1, stretches them into the interval $[l, r]$, and finally clips them back to $[0, 1]$:

$$\mathbf{u} \sim \mathcal{U}(0, 1), \quad \mathbf{s} = \text{Sigmoid}((\logit(\mathbf{u}) + \log \alpha)/\beta), \quad \bar{\mathbf{s}} = \mathbf{s}(r - l) + l, \quad (2)$$

$$\mathbf{z} = \min(1, \max(0, \bar{\mathbf{s}})). \quad (3)$$

Here, $\alpha \in \mathbb{R}^{d_2}$ are learnable parameters and $\beta > 0$ is the temperature. Clipping sets values below 0 to exactly 0 and values above 1 to exactly 1. This makes the decision discrete by allowing some entries to be exactly switched on or off. This properties of having discrete decisions in a fully differentiable way is a key property since it allows to skip computation where $\mathbf{z}_i = 0$. This approach has proven effective for learning static pruning mask (Xia et al., 2024; Wang et al., 2020)

In order to learn predictive activation sparsity, we need adapt this mechanism. First, it was designed to learn a fixed pruning mask, while we want to predict which coordinates to zero out based on the current input token. Second, we want the active gates to remain continuous rather than being clipped to 1. To address the first point, we make α the output of an input-dependent network. To address the second, we apply stretching only on the left boundary (0). Given an input $\mathbf{x} \in \mathbb{R}^{d_1}$, we define ZeroNeurone $\text{ZoN}(\mathbf{x})$, a revised hard-concrete distribution, as:

$$\mathbf{u} \sim \mathcal{U}(0, 1), \quad \mathbf{s} = \text{Sigmoid}((\logit(\mathbf{u}) + \log \alpha(\mathbf{x}))/\beta), \quad \bar{\mathbf{s}} = \mathbf{s} + l(1 - \mathbf{s}), \quad (4)$$

$$\mathbf{z} = \max(0, \bar{\mathbf{s}}). \quad (5)$$

It is possible to encourage sparsity using l_0 regularization which is the expected number of active output coordinates and has a differentiable closed form:

$$\mathbb{E}[\|\mathbf{z}\|_0] = \sum \text{sigmoid}(\log \alpha(x) - \beta \log -l). \quad (6)$$

Adding $\mathbb{E}[\|\mathbf{z}\|_0]$ to the training loss encourages fewer active coordinates. Since minimizing $\mathbb{E}[\|\mathbf{z}\|_0]$ optimizes directly the amount of non-active coordinates, the desired target active intermediate dimension h can be constrained with lagrangians:

$$\mathcal{L}_{\text{sparsity}} = \lambda_1(\mathbb{E}[\|\mathbf{z}\|_0] - h) + \lambda_2(\mathbb{E}[\|\mathbf{z}\|_0] - h)^2 \quad (7)$$

where λ_1 and λ_2 are the trainable lagrangian scalar parameters.

Listing 1: Minimal PyTorch implementation of the ZoN MLP training forward pass.

```

def mlp_zon(x, W_k, W_v, act_fn, a, beta=1.0, l=-0.01):
    """
    x: [batch * seq_len, d_model]
    K: [d_model, d_inter]
    V: [d_inter, d_model]
    a: gate network, d_model -> d_inter
    """
    log_alpha = a(x) # shape: [batch * seq_len, d_inter]

    # Sample gates
    u = torch.empty_like(log_alpha).uniform_(eps, 1 - eps)
    s = torch.sigmoid((torch.logit(u) + log_alpha) / beta)
    # stretch beyond zero and clamp to 0.0
    z = (s + l * (1 - s)).clamp(min=0.0)
    # Expected number of active gates per token
    num_active = torch.sigmoid(log_alpha - beta * math.log(-l)).sum(-1)

    h = act_fn((x @ W_k) * z)
    o = h @ W_v

    return o, num_active

```

Overall, this sparse gating inherits from some nice properties of hard-concrete distribution. The sparsity constraints directly optimizes the amount of non-active coordinate in a differentiable way. Since the gating is based sigmoid followed by a stretching, the sparsification process is fully point-wise which is beneficial in tensor-parallelism scenario as we will discuss.

3.2 STRUCTURED ACTIVATION SPARSITY OF THE MLP

Recalling the MLP definition as key-value memory, its sparsily activated version can be written as:

$$\text{MLP}_{\text{ZoN}}(\mathbf{x}) = \sigma((\mathbf{x}\mathbf{W}_k) \odot \mathbf{z})\mathbf{W}_v \quad (8)$$

where $\mathbf{z} \in \mathbb{R}^{d_{\text{inter}}}$ is the output of the activation sparsity gating $\mathbf{z} = \text{ZoN}(\mathbf{x})$. Since $\mathbf{z}$ predicts ahead of time the coordinates d_{inter} to zero out for this input token, it decides which key-value pairs to ignore. Hence, the sparsity is structured and can be leveraged to accelerate training provided it exists a GPU kernel that can leverage this structured sparsity. For the predicted sparsity to be used for sparse computation in both $\mathbf{W}_k$ and $\mathbf{W}_v$, the non-linear activation σ must preserve zeros: $\sigma(0) = 0$. This ensures that coordinates zeroed out by the gate remain zero after the activation. Fortunately, this holds for common activations in modern transformer MLPs, such as SiLU, ReLU, and ReLU². Figure 1 provides an implementation of the sparsely activated MLP in PyTorch.

3.2.1 LOCAL VS GLOBAL SPARSITY CONSTRAINT

While the provided implementation is illustrated on a single MLP layer, the transformer model has L layers of such MLP. Each of them has its own sparse gating. The sparsity constraint can be applied either locally or globally. When apply the sparsity constraint locally, each MLP layer converges to the specified target sparsity. In contrast, applying the constraint globally means that only the average sparsity across layers is constrained. Individual layers can therefore use different sparsity levels, allowing the model to allocate more active neurons to layers that require greater capacity while inducing higher sparsity in less demanding layers. This provides greater flexibility while maintaining the desired overall compute budget. In our setting, we use global sparsity constraint at it gives more flexibility. Hence we define the average expected of active gates across all MLP layers L as $\mathbb{E}_L[\|\mathbf{Z}\|_0] = \frac{1}{L} \sum_{l=1}^L \mathbb{E}_l[\|\mathbf{z}\|_0]$. Putting $\mathbb{E}_L[\|\mathbf{Z}\|_0]$ in Equation 7 as expected sparsity, will optimize the constrained global sparsity across all layers.

3.2.2 PARAMETRIZING THE GATE NETWORK

The design of the gate network $\alpha(\mathbf{x})$ is free and can be made complex, non-linear and deep. However, the design must scale its number of parameters as the intermediate dimension size scale. We found that the gating network can be very small without hurting performance. So we constrain its rank to $\alpha(\mathbf{x}) = \alpha_1 \alpha_2(\mathbf{x})$, with $\alpha_1 \in \mathbb{R}^{d_{\text{model}} \times r}$ and $\alpha_2 \in \mathbb{R}^{r \times d_{\text{model}}}$. We found a small rank is sufficient, so we fix in our experiments $r = d_{\text{model}}/8$. We do not yet understand why such small gating networks work well. One possible explanation is that the sigmoid is applied independently to each coordinate, so each gating decision can be made without modeling interactions among all intermediate dimensions. In contrast, a softmax produces a probability distribution jointly over the intermediate dimensions, which may require a larger routing network to model these dependencies.

3.2.3 ON THE EFFICIENCY OF ZON

The proposed sparse activation gating accelerates the training and inference over dense models in different ways.

First, the approach is a trainable structured activation sparsity. The sparsity emerges quickly during training and a sparse GPU kernel can exploit this to skip computation of the zero-out coordinates to accelerate training and inference. Given $\mathbf{z} \in \mathbb{R}^{t \times d_{\text{inter}}}$ the predicted sparse gating, the corresponding predicted zero gates in W_k and W_v can be skipped during the MLP forward. There are many sparse GPU kernels in the literature that can be adapted to accelerate training and inference, such as DejaVu or TEAL.

Second, the proposed sparsification approach is pointwise: the sigmoid and stretching apply independently to each coordinate. Compared to TopK-based activation sparsity, this makes the approach less communication bound in distributed training, specially in tensor parallelism sharding. Suppose the gates and the two MLP projections are sharded along the intermediate dimension across N GPUs. Each GPU predicts its local gates from the same input $\mathbf{x}$ and applies `zon_mlp` using its local weights to compute its contribution $\mathbf{y}_i$ to the output. An all-reduce then sums these contributions across GPUs and gives each GPU the full output $\mathbf{y} = \sum_{i=1}^N \mathbf{y}_i$. This is the same output communication used in a dense tensor-parallel MLP. This approach uses therefore less communication than TopK-based sparse memory layer that needs to dispatch token representations to the GPU hosting their selected memory slots.

The main possible disadvantage of the proposed approach is that different tokens can activate different MLP intermediate coordinates, leading to scattered memory accesses for sparse computation. This can make the computation memory-bandwidth-bound: reading the weights becomes the bottleneck rather than performing the computation. One way to reduce this cost is to group tokens that activate similar coordinates into buckets and process them together. The benefit depends on how much their active coordinates overlap and the cost of grouping the tokens.

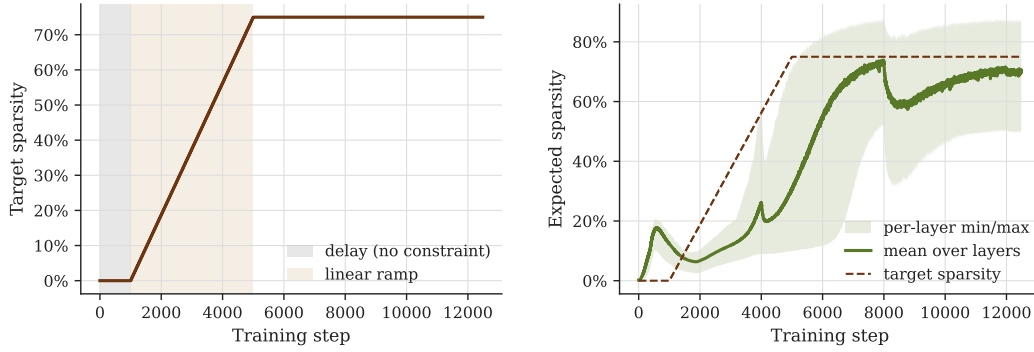
4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP.

Implementation. Our experiments are based on nanochat, a minimalist GPT implementation optimized for dense training. For fair comparisons, we keep unchanged the hyperparameters settings, which has been optimized for dense models. Transformer matrices, including the sparse gate network projections, are optimized with Muon. The embeddings, the output head, biases, and scalar parameters use AdamW. In each MLP, the gate network uses two linear projections with a bottleneck of $d_{\text{model}}/8$ and a learned output bias. We set $\beta = 1$ and $l = -0.01$. The gates are initialized to keep almost all coordinates active at the beginning of the training. The gradient signal of the sparsity loss can be as strong as the language modeling loss at the beginning of training. So, we introduce a sparsity rampup where we leave sparsity unconstrained for the first 1,000 steps, then linearly increase the target sparsity over 4,000 steps. The constraint is applied to the average expected sparsity across layers, with a Lagrange multiplier updated at each training step. Gates are sampled during training and computed deterministically at evaluation.

Table 2: Model architecture and training budgets. Dense-600M uses a narrower MLP; the other architecture settings are shared.

Configuration	Dense-600M	Dense-1.2B	ZoN-1.2B-A600M
Total parameters	600,834,122	1,233,125,450	1,233,125,450
Transformer layers		24	
Hidden dimension		1,536	
Query heads		12	
Key/value heads		12	
MLP expansion multiplier	2.417×		8×
MLP intermediate dimension	3,712		12,288
Vocabulary size		32,768	
Context length		2,048	
Sliding-window pattern		3 : 1	
Training tokens (approx.)		13B	



(a) Target sparsity schedule with the delay and linear ramp phases.

(b) The convergence of the expected sparsity.

Figure 2: Training dynamics of the adaptive ZoN gate.

Models. We train three models: Dense-600M, a 600M dense model, Dense-1.2B, a 1.2B dense model, and ZoN-1.2B-A600M a 1.2B dense model with 600M active parameters using our proposed approach. All three models are trained on approximately 13B tokens, with their architecture settings summarized in Table 2. Dense-1.2B uses the same MLP dimensions as ZoN-1.2B-A600M, while Dense-600M uses a narrower MLP to match its approximate active parameter count. This setup helps us study whether at same active parameters, increasing the total model parameters improves the performance.

Evaluation. We follow the CORE evaluation setup from DataComp-LM (Li et al., 2025), as implemented in nanochat. The suite contains 22 tasks covering commonsense reasoning, reading comprehension, language modeling, knowledge, and symbolic reasoning, with task-specific prompts and few-shot settings. The evaluator scores multiple-choice and schema tasks by selecting the answer with the highest average log-probability over the scored continuation. For completion tasks, every target token must be the model’s most likely prediction given the preceding reference tokens. We report raw task accuracies and the aggregate CORE score. To compute CORE aggregated score, each task accuracy a is first centered as $(a - b)/(1 - b)$, where b is the task’s random baseline.

4.2 RESULTS

Table 3 shows that ZoN-1.2B-A600M reaches a CORE score of 29.77, compared with 29.61 for Dense-1.2B and 27.35 for Dense-600M. ZoN therefore achieves similar aggregate performance to the larger dense model while using roughly 600M active parameters per token. Compared with the dense model of similar active size, it improves CORE by 2.42 percentage points. This supports

Table 3: Benchmark accuracy (%) for the dense and sparse models. CORE reports the mean centered score (%). BB denotes BIG-bench.

Category	Benchmark (n-shot)	Dense-600M	Dense-1.2B	ZoN-1.2B-A600M
<i>Commonsense reasoning</i>	PIQA (10)	74.37	76.93	75.35
	WinoGrande (0)	56.91	60.30	60.77
	ARC-Easy (10)	69.40	72.10	70.67
	ARC-Challenge (10)	40.10	42.66	40.96
	HellaSwag (0)	57.02	61.79	60.52
	HellaSwag (10)	57.24	62.05	61.24
<i>Commonsense reasoning and reading comprehension</i>	COPA (0)	66.00	64.00	68.00
	CommonsenseQA (10)	21.38	32.02	28.67
	OpenBookQA (0)	40.40	43.20	41.40
	Winograd (0)	69.23	68.13	73.63
	BoolQ (10)	61.62	63.79	61.31
	SQuAD (10)	45.69	46.34	48.20
<i>Reading comprehension, language modeling, and knowledge</i>	CoQA (0)	33.30	34.22	35.04
	LAMBADA (OpenAI) (0)	43.22	46.81	46.96
	Jeopardy (10)	9.40	15.21	15.07
	BB: QA Wikidata (10)	46.44	51.30	50.51
	BB: Language identification (10)	24.98	24.77	24.81
	AGI Eval: LSAT-AR (3)	26.96	22.17	22.17
<i>Symbolic reasoning</i>	BB: Dyck languages (10)	12.20	13.10	11.60
	BB: CS algorithms (10)	44.32	39.39	44.32
	BB: Operators (10)	18.57	17.62	19.05
	BB: Repeat copy logic (10)	3.12	0.00	0.00
CORE		27.3510	29.6130	29.7722

the idea that keeping a larger memory and selecting which parts to activate in the intermediate dimension can be more effective than using a smaller dense memory.

The comparison with Dense-600M is consistent across many tasks. For example, ZoN improves HellaSwag accuracy from 57.24% to 61.24%, CommonsenseQA from 21.38% to 28.67%, and QA Wikidata from 46.44% to 50.51%. Compared with Dense-1.2B, ZoN performs better on Winograd and SQuAD, but worse on PIQA, ARC-Challenge, and CommonsenseQA. Hence, the small CORE difference of 0.16 percentage points between the dense and sparse 1.2B models reflects comparable overall performance rather than a consistent advantage over the larger dense model.

As described above, we gradually increase the target sparsity during training to let the model focus on language modeling at the beginning. Figure 2a shows this schedule, while Figure 2b shows that the expected sparsity closely follows it. This suggests that suitable sparse kernels could accelerate training once sparsity emerges. For longer training runs, the same delay and ramp would account for a smaller fraction of the total training steps. The sparsity constraint applies to an average over input tokens, so individual tokens and tasks can have different sparsity levels. The reported sparsity of approximately 70% is an average across evaluation tasks. Table 4 reports the predicted sparsity (Zero frac.) for each task, along with the centered accuracies used to compute the CORE metric.

5 GENERALIZATION TO EMBEDDING LANGUAGE MODELS

We apply the same principle to embedding language models, using ZoN to predict which coordinates of an embedding to keep. Here, we use NV-Embed representations and train the gates without a specific target sparsity constraint. Unlike the MLP setting, we stretch on both the left and right sides and clip the gates to $[0, 1]$, encouraging gates to be either zero or one. This saves disk memory as for vector store, binary gates are cheaper. At inference, we keep the k coordinates with the highest sigmoid gate scores, with $k \in \{128, 256\}$ for evaluation, but k can be chosen freely during inference, depending on the budget.

Table 4: Evaluation of the dense and ZoN sparse runs. All entries are percentages; CORE is the mean centered score. %0 measures the fraction of zero ZoN MLP gates in the sparse run. BB denotes BIG-bench.

Task	Accuracy		Centered		Predicted %0.
	Dense	Sparse	Dense	Sparse	Sparse
HellaSwag (zero-shot)	61.79	60.52	49.05	47.35	69.19
Jeopardy	15.21	15.07	15.21	15.07	65.20
BB: QA Wikidata	51.30	50.51	51.30	50.51	65.30
ARC-Easy	72.10	70.67	62.79	60.89	69.56
ARC-Challenge	42.66	40.96	23.55	21.27	69.40
COPA	64.00	68.00	28.00	36.00	67.41
CommonsenseQA	32.02	28.67	15.03	10.83	71.84
PIQA	76.93	75.35	53.86	50.71	69.14
OpenBookQA	43.20	41.40	24.27	21.87	68.05
LAMBADA (OpenAI)	46.81	46.96	46.81	46.96	66.17
HellaSwag	62.05	61.24	49.40	48.32	68.29
Winograd	68.13	73.63	36.26	47.25	67.09
WinoGrande	60.30	60.77	20.60	21.55	66.98
BB: Dyck languages	13.10	11.60	13.10	11.60	75.95
AGI Eval: LSAT-AR	22.17	22.17	2.72	2.72	72.12
BB: CS algorithms	39.39	44.32	39.39	44.32	73.14
BB: Operators	17.62	19.05	17.62	19.05	73.53
BB: Repeat copy logic	0.00	0.00	0.00	0.00	70.89
SQuAD	46.34	48.20	46.34	48.20	67.87
CoQA	34.22	35.04	34.22	35.04	68.45
BoolQ	63.79	61.31	4.72	-1.80	67.76
BB: Language identification	24.77	24.81	17.24	17.28	67.62
CORE	—	—	29.6130	29.7722	—

Table 5: Performance of NV-Embed representations on classification and retrieval tasks. Scores are percentages. k is the number of coordinates retained at inference; dense representations use all d_{model} coordinates. The dense baseline uses NV-Embed with a projector fine-tuned on each task and retains all coordinates. The best score in each column is bold and the second-highest distinct score is underlined.

			Classification		Retrieval	
			Top-1 Acc. (%) $\uparrow$		NDCG@10 (%) $\uparrow$	
Category	Method	Dim.	MTOPIntent	Banking77	NFCorpus	SciFact
Dense	Fine-tuned	d_{model}	<u>93.96</u>	<u>93.73</u>	55.01	84.87
Sparse	CSR	256	96.85	94.38	56.12	73.20
	CSR	128	96.85	94.38	54.64	73.20
	ZoN (ours)	256	91.68	92.93	<u>55.46</u>	<u>82.06</u>
	ZoN (ours)	128	90.04	91.02	52.14	81.23

Experimental setup. For retrieval, we train a linear projector of shape $d_{\text{model}} \times d_{\text{model}}$ on top of the embeddings, alongside the gate projector. For classification, we instead use two a perceptron, with linear projectors d_{model} to $4d_{\text{model}}$ and back to d_{model} , with a ReLU activation between them. In both settings, the gate network uses a hidden dimension of $r = d_{\text{model}}/4$, with d_{model} being 4096 for NV-Embed. We evaluate retrieval on NFCorpus and SciFact using NDCG@10, and classification on MTOPIntent and Banking77 using top-1 accuracy. We train for 10 epochs on each dataset, except NFCorpus, where we use 2 epochs due to the training budget.

Results. Table 5 shows that the benefit depends on the task. On SciFact, ZoN outperforms CSR, but remains below the full fine-tuned representation. On NFCorpus, ZoN with $k = 256$ slightly exceeds the full fine-tuned representation, while CSR is better overall. For classification, ZoN re-

mains below both CSR and the baseline using NV-Embed with a projector fine-tuned on the task and all embedding coordinates retained. Increasing k from 128 to 256 improves ZoN on all four tasks. These results suggest that adaptive coordinate selection can preserve much of the retrieval performance.

REFERENCES

- Damai Dai, Li Dong, Yaru Hao, Zhifang Sui, Baobao Chang, and Furu Wei. Knowledge neurons in pretrained transformers, 2022. URL <https://arxiv.org/abs/2104.08696>.
- Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 5484–5495, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.446. URL <https://aclanthology.org/2021.emnlp-main.446/>.
- Xu Owen He. Mixture of a million experts, 2024. URL <https://arxiv.org/abs/2407.04153>.
- Guillaume Lample, Alexandre Sablayrolles, Marc’Aurelio Ranzato, Ludovic Denoyer, and Herve Jegou. Large memory layers with product keys. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché Buc, E. Fox, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/9d8df73a3cfbf3c5b47bc9b50f214aff-Paper.pdf.
- Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, Sedrick Keh, Kushal Arora, Saurabh Garg, Rui Xin, Niklas Muennighoff, Reinhard Heckel, Jean Mercat, Mayee Chen, Suchin Gururangan, Mitchell Wortsman, Alon Albalak, Yonatan Bitton, Marianna Nezhurina, Amro Abbas, Cheng-Yu Hsieh, Dhruva Ghosh, Josh Gardner, Maciej Kilian, Hanlin Zhang, Rulin Shao, Sarah Pratt, Sunny Sanyal, Gabriel Ilharco, Giannis Daras, Kalyani Marathe, Aaron Gokaslan, Jieyu Zhang, Khyathi Chandu, Thao Nguyen, Igor Vasiljevic, Sham Kakade, Shuran Song, Sujay Sanghavi, Fartash Faghri, Se-woong Oh, Luke Zettlemoyer, Kyle Lo, Alaaeldin El-Nouby, Hadi Pouransari, Alexander Toshev, Stephanie Wang, Dirk Groeneveld, Luca Soldaini, Pang Wei Koh, Jenia Jitsev, Thomas Kol-
lar, Alexandros G. Dimakis, Yair Carmon, Achal Dave, Ludwig Schmidt, and Vaishaal Shankar. DataComp-LM: In search of the next generation of training sets for language models, 2025. URL <https://arxiv.org/abs/2406.11794>.
- Christos Louizos, Max Welling, and Diederik P. Kingma. Learning sparse neural networks through L_0 regularization. In *International Conference on Learning Representations*, 2018. URL <https://arxiv.org/abs/1712.01312>.
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT, 2023. URL <https://arxiv.org/abs/2202.05262>.
- Ziheng Wang, Jeremy Wohlwend, and Tao Lei. Structured pruning of large language models. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6151–6162, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.496. URL <https://aclanthology.org/2020.emnlp-main.496/>.
- Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. Sheared LLaMA: Accelerating language model pre-training via structured pruning. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=09iOdaeOzp>.

A APPENDIX

You may include other additional sections here.